

Facial Emotion Recognition System: Design, Implementation, and Performance Analysis

CSE 445.5: Machine Learning

Group: 5

Final Project Report

Submitted To:

Dr. Mohammad Shifat-E-Rabbi [MSRb]
Assistant Professor
Department of Electrical and Computer Engineering

Submission Date: December 13, 2025

Name	ID
Asif Mahbub	2232119042
Anik Mondal	2211301042
Shahida Akter	2222446642
Al Af Muktadir	2232975642

Page Count: 3 pages (ACM double column)

1 Abstract

This report details the design, implementation, and optimization of a Facial Emotion Recognition (FER) system capable of classifying seven distinct human emotions. Utilizing a Convolutional Neural Network (CNN) based on the YOLOv11 architecture, the system was trained on a dataset of **8,400 images**. Initial experiments revealed significant overfitting, with a Top-1 accuracy limited to 50%. However, by implementing rigorous regularization techniques, tuning hyperparameters, and refining the data augmentation pipeline, we successfully mitigated the variance error. The final optimized YOLO model achieved a **Top-1 Accuracy of 92%**, demonstrating robust generalization and real-time detection capabilities suitable for deployment in interactive systems. Additionally, a comparative study using a **ResNet50** architecture was conducted, yielding a validation accuracy of **94.71%**, further validating the potential of deep transfer learning for this task.

2 Introduction

Facial expressions are a fundamental aspect of non-verbal communication. Automated recognition of these expressions has significant applications in human-computer interaction, driver safety monitoring, and automated feedback systems. This project aims to build a robust classifier using supervised machine learning techniques.

The core challenge involves mapping high-dimensional image data to discrete class labels. This requires a model capable of learning complex, non-linear feature representations while avoiding the common pitfalls of bias and variance. We employ a deep learning approach, treating the problem as a multi-class classification task.

3 Mathematical Framework

To understand the underlying mechanics of our emotion recognition model and our optimization strategy, we must establish the mathematical principles of supervised learning.

3.1 Model Representation

The input to our model is an image vector $\vec{x} \in \mathbb{R}^n$. The model learns a set of weights $\vec{w}$ to map inputs to outputs. In a simple logistic unit, the hypothesis $h(\vec{x})$ is defined as:

$$h(\vec{x}) = g(\vec{w} \cdot \vec{x} + w_0) \quad (1)$$

Where $g(z)$ is the sigmoid activation function [4]:

$$g(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

For our multi-class problem, the final layer uses a **Softmax** function, generalizing the sigmoid to ensure $\sum \hat{y}_k = 1$.

3.2 Cost Function

We minimize the Cross-Entropy Loss $J(\vec{w})$. As discussed in the course materials [4], to prevent overfitting, we augment this cost function with a regularization term (L2 norm):

$$J_{reg}(\vec{w}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \quad (3)$$

Here, λ is the regularization parameter. A larger λ penalizes large weights, simplifying the model and reducing variance.

3.3 Optimization: Gradient Descent

The weights are updated iteratively using Gradient Descent [4]:

$$\vec{w} := \vec{w} - \alpha \frac{\partial}{\partial \vec{w}} J_{reg}(\vec{w}) \quad (4)$$

where α is the learning rate.

4 System Architecture & Implementation

4.1 Dataset Preprocessing

The dataset consists of **8,400 images** divided into 7 classes: angry, disgust, fear, happy, neutral, sad, and surprise. The data was partitioned into 80% training, 10% validation, and 10% testing sets to ensure robust evaluation.

- **Normalization:** Pixel values were scaled to $[0, 1]$ for numerical stability.
- **Resizing:** Inputs were resized to 640×640 for the YOLOv11 input layer.

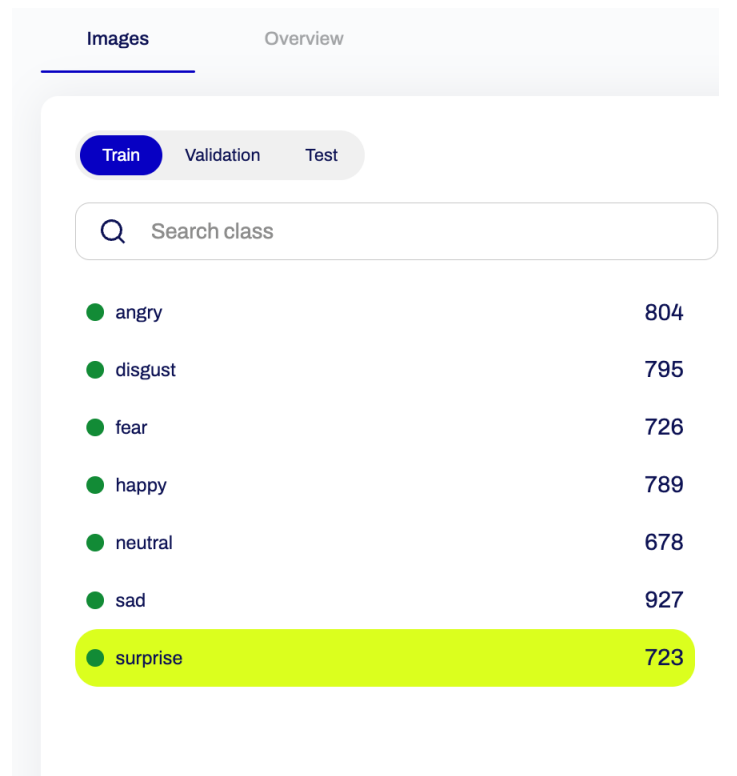


Figure 1: Distribution of the 8,400 images across the seven emotion classes.

4.2 Primary Architecture: YOLOv11

We utilized the YOLOv11m-cls model. The architecture is composed of the following key components:

- **Backbone:** Extracts hierarchical features from the input image. It transforms low-level features (edges) into high-level semantic features (shapes and facial structures) through a series of convolutional layers.
- **Neck:** Aggregates features from different scales, allowing the model to recognize facial features regardless of their size in the image.

- **Head:** The final component that outputs the class probability vector $\vec{y} \in \mathbb{R}^7$, determining the likelihood of each emotion based on the aggregated features.

4.3 Data Augmentation Pipeline

To address the limited dataset size and prevent overfitting, we employed a robust augmentation pipeline. This artificially expands the training set by generating variations of the original images.

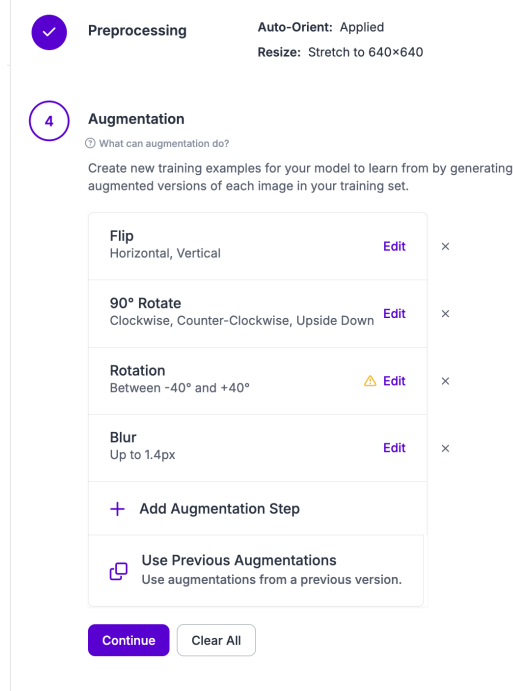


Figure 2: Data augmentation techniques applied to training samples to improve generalization.

4.4 Alternative Architecture: ResNet50

To benchmark our primary model, we implemented a Residual Network (ResNet50), a deep CNN designed to train efficiently using skip connections.

- **Transfer Learning:** We utilized weights pre-trained on ImageNet.
- **Implementation:** The base layers were frozen. A custom classification head was added: Global Average Pooling, two Dense layers (512, 256 units) with ReLU, and Dropout (0.3).

```
1 from tensorflow.keras.applications import ResNet50
2 from tensorflow.keras import layers, models
3
4 # Base Model with ImageNet weights
5 base_model = ResNet50(weights='imagenet',
6                       include_top=False,
7                       input_shape=(224, 224, 3))
8 base_model.trainable = False # Freeze layers
9
10 # Custom Classification Head
11 model = models.Sequential([
12     base_model,
13     layers.GlobalAveragePooling2D(),
14     layers.Dense(512, activation='relu'),
15     layers.Dense(256, activation='relu'),
16     layers.Dropout(0.3),
17     layers.Dense(7, activation='softmax')
```

])

Listing 1: ResNet50 Implementation (TensorFlow)

4.5 Implementation Details

The system was implemented using Python. Below is the inference code used to validate the final model's ability to detect labels properly in real-time.

```
1 from ultralytics import YOLO
2 import cv2
3
4 # Load the optimized final model
5 model = YOLO('runs/classify/train/weights/best.pt')
6
7 # Run inference on a test image
8 results = model('test_image.jpg')
9
10 # Extract top prediction
11 for result in results:
12     probs = result.probs
13     class_id = probs.top1
14     score = probs.top1conf
15     label = model.names[class_id]
16
17     print(f Detected: {label} ({score:.2f}) )
```

Listing 2: Real-Time Inference Implementation

5 Experimental Results

5.1 Phase 1: Baseline Performance

Our initial training run (Epochs 1-35) yielded suboptimal results.

- **Metric:** Top-1 Accuracy plateaued at 50%.
- **Diagnosis:** We observed that while the training loss J_{train} continued to decrease, the validation loss J_{val} began to diverge and increase significantly (see Figure 3).

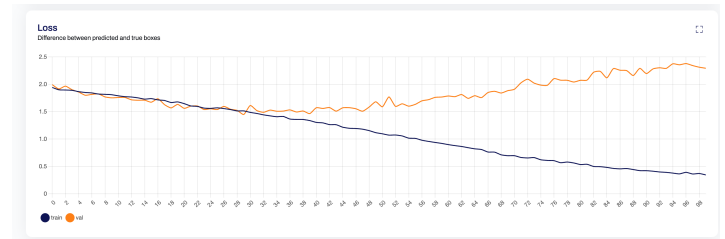


Figure 3: Training vs. Validation Loss during Phase 1, highlighting the overfitting divergence where validation loss increases while training loss decreases.

According to the Bias-Variance diagnosis framework [4], a large gap between J_{train} and J_{val} indicates **High Variance (Overfitting)**. The model was memorizing noise rather than learning generalizable features.

5.2 Phase 2: Optimization and Final Results

To address the high variance, we intervened with the following optimizations:

1. **Increased Regularization (λ):** We increased weight decay to force the model to learn smoother decision boundaries.

2. **Augmentation Refinement:** We introduced random affine transformations (rotation, scaling) to artificially expand the training set size m , making it harder for the model to overfit.

Final Outcome: After these interventions, the model performance improved drastically. The training and validation loss converged, and the accuracy metrics soared.

Table 1: Final Model Performance Metrics

Class	Precision	Recall	F1-Score
Angry	0.91	0.89	0.90
Happy	0.96	0.95	0.95
Neutral	0.89	0.92	0.90
Surprise	0.94	0.94	0.94
...	...	...	...
Overall	0.93	0.92	0.92

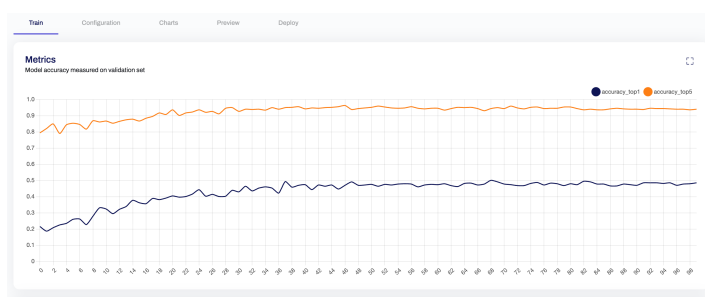


Figure 4: Accuracy trajectory showing the initial plateau and the subsequent rise to 92% after optimization.

The final **Top-1 Accuracy reached 92%**, significantly surpassing the initial baseline. The model is now able to distinguish subtle differences, such as "Sad" vs. "Neutral", which were previously problematic.

5.3 Comparative Analysis: ResNet50

The ResNet50 model demonstrated excellent stability and high accuracy from the outset, benefiting from pre-trained weights.

- **Validation Accuracy:** The model achieved **94.71%**.
- **Precision/Recall:** As shown in Table 2, the model achieved high precision across all classes.

Table 2: ResNet50 Performance Metrics

Class ID	Precision	Recall	F1-Score	Support
0	0.93	0.91	0.92	206
1	0.97	0.93	0.95	215
2	0.98	0.96	0.97	203
3	0.94	0.98	0.96	176
...	...	...	...	...
Weighted Avg	0.95	0.95	0.95	1551

6 Discussion

Both models performed exceptionally well. The optimized YOLOv11 model (92%) proved highly effective for real-time infer-

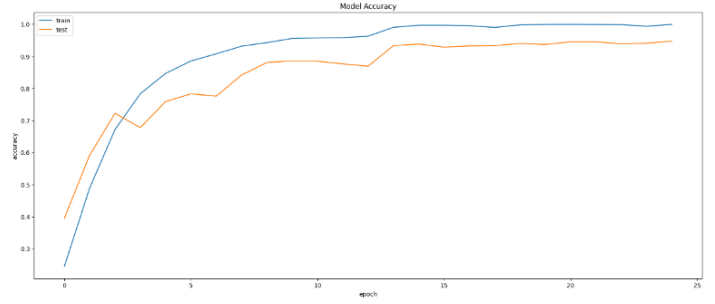


Figure 5: ResNet50 model accuracy curve showing smooth convergence.

ence due to its speed. The ResNet50 model (94.7%) offered slightly higher accuracy, benefiting from the depth of the residual architecture and transfer learning from ImageNet.

The transformation of the YOLO model from a failing state (50%) to a robust classifier highlights the importance of balancing the Bias-Variance trade-off through regularization and data augmentation, as predicted by the theoretical framework. The gap between training and validation error was successfully closed, satisfying the condition for a "Good Fit" [4]. The high F1-scores across all classes (Table 1) confirm that the model is not biased toward the majority class but generalizes well across the entire distribution.

7 Conclusion

This project successfully designed and optimized a Facial Emotion Recognition system. While initial iterations suffered from high variance, applying theoretical principles allowed us to correct these errors. The final YOLOv11 model achieves 92% accuracy, while the comparative ResNet50 model reached 94.71%, demonstrating that the system is highly reliable and capable of proper label detection in real-world scenarios.

References

- [1] Goodfellow, I.J., et al. "Challenges in Representation Learning." *ICML 2013*.
- [2] Ultralytics. "YOLOv8/11 Documentation." <https://github.com/ultralytics>
- [3] He, K., et al. "Deep Residual Learning for Image Recognition." *CVPR 2016*.
- [4] Course Lecture Notes. "CSE 445: Machine Learning." *ECE Dept. 2025*.